

Rapport de Projet : Sokoban

Esono Mangué Ainhoa
Lacoste Elowan

L2 info 2
L2 info 5



Afin de développer notre Sokoban nous avons suivi les différentes étapes du projet et réglé au cas par cas les différentes fonctionnalités. Une fois les exercices préliminaires donnés en partie 1 terminés nous nous sommes inspirés d'eux afin de commencer la partie 2 du projet.

Tout d'abord il a fallu créer la classe Sokoban(object) pour pouvoir lancer le jeu, avec la bibliothèque Tkinter, et l'utilisation de SokobanXSBLLevels stockant les matrices des différents niveaux, nous pouvons afficher la fenêtre de jeu avec la méthode play().

Ensuite WharehousePlan(object); C'est le plan de l'entrepôt. Les éléments sont stockés dans une matrice (#rawMatrix).

Classe WharehousePlan :

- appendRow()
 - Elle nous permet d'ajouter une ligne à la matrice
- at()
 - permet de retourner l'élément à la position donnée
- atPut()
 - Prend un objectif position et un objectif elem comme arguments et met à jour elem à la position spécifiée dans la matrice
- isPositionValid()
 - Regarde si la position donnée est dans les limites de la matrices.
- hasFreePlaceAt()
 - Regarde si la position donnée est libre(pas de box, pas de goal, pas le joueur)

Par la suite nous avons complété la classe Mover(object) et la classe Level(object), le but était de pouvoir déplacer le personnage et qu'en fonction de la direction dans laquelle il allait, l'aperçu était modifié à l'aide de quatre images différentes (personnage vers le haut, le bas, la gauche, la droite).

Classe Mover :

- moveInCanvas()
 - Permet de déplacer le player dans la matrice
- def canMove()
 - détermine s'il y a mouvement du player ou pas par rapport aux obstacles
- setupImageForDirection()

- Permet de changer l'image du player en fonction de la direction

Classe Level :

- `initWharehouseFromXsb()`
 - Permet d'appeler les différentes classes et d'afficher tous les éléments du jeu.
- `keypressed()`
 - Permet de définir le mouvement gauche ,droite, en haut, en bas avec les touches du clavier

Après il aurait fallu pouvoir déplacer les caisses, Pour se faire la classe `Position(object)` et la classe `Box(object)` aurait été utile.

Classe Position :

- `positionTowards()`
 - retourne la position vers la direction en tenant compte de l'offset, c'est à dire pas la position juste devant le mover mais celle d'un cran après là où se trouverait la caisse une fois poussée.

Classe Box :

- `moveTowards()`
 - Permet de bouger la box dans une direction spécifique (en haut, en bas, à gauche, à droite)

Pour finir nous voulions ajouter la classe `Score (object)`, cette dernière devait comptabiliser le nombre de déplacements du joueurs et l'inscrire dans un fichier json.

Classe Score:

- Nous avons la méthode déplacement permettant de retourner le nombre de déplacements et plus le joueur se déplace plus il gagne des points.

Les dernières classes : Floor(object) ; Goal(object) ; Wall(object) ainsi que les classes précédemment citées : Box(object) ; Mover(object) ont dans leurs `__init__` l'image qui doit être affichée.

Pour conclure nous avons remis plusieurs versions du projet, chacune abordant différents points.

- ❖ PCOFinal.ipynb correspond à la version dans laquelle le joueur peut se déplacer dans l'unique limite des murs, il peut traverser les caisses sans qu'elles ne disparaissent pareil pour les "Goal".
- ❖ Direction.ipynb correspond à la version dans laquelle le joueur change de visuel en fonction de la direction prise, problème : il disparaît lorsqu'il rencontre un mur.
- ❖ Box.ipynb correspond à la version dans laquelle le joueur peut se déplacer sans pouvoir traverser ni les murs ni les caisses (les caisses ne peuvent pas être déplacées)
- ❖ Score.ipynb : Début de code pour avoir un score; comptage des mouvements dans une variable lorsqu'on appuie sur une touche de déplacement et création de la classe Score, non-abouti.

Avec plus de temps nous aurions pu rendre une seule version contenant toutes les fonctionnalités du jeu, on a passé beaucoup de temps à essayer de déplacer les caisses sans réussir. Pour améliorer le jeu nous avons imaginé :

1. Pouvoir choisir le niveau de difficulté avant de commencer
2. Un affichage du score sur la fenêtre
3. Peut-être pouvoir choisir son personnage avant de commencer

